Jawaharlal Nehru Technological University Hyderabad

University College of Engineering, Science & Technology

Real-time / Field-Based Research Project Report On

Orbital Debris Avoidance & Constellation Management System

Autonomous Space Situational Awareness (SSA) Dashboard

Submitted by
Subhasish Budati (24011P0510)
Mohit Pradhan (24011P0532)
K. Mohan Murari (24011P0541)
V. Pramodh (24011P0550)
S. Sooraj (24011P0566)

Department of Computer Science and Engineering
Academic Year 2025-2026

Certificate

This is to certify that the project work entitled "Orbital Debris Avoidance & Constellation Management System" an Autonomous Space Situational Awareness (SSA) Dashboard was developed and submitted as part of Real time research project, Jawaharlal Nehru Technological University, Hyderabad.

The project was carried out for the Real-time / Field-Based Research Project during the academic year 2025-2026, under the Department of Computer Science and Engineering , JNTUH.

Professor and Head, CSE	**Project Supervisor**
(Dr. Santhi sree)	(Mr. G Narasimham)

Submitted for Autonomous End Semester Examination Mini Project Viva-Voce on

_____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

Abstract

The rapid proliferation of satellites and uncontrolled space debris in Low Earth Orbit (LEO) has created an increasingly hazardous environment that conventional, ground-reliant collision avoidance systems are ill-equipped to handle at scale. Manual evaluation of Conjunction Data Messages (CDMs) by Flight Dynamics Officers cannot scale to the thousands of conjunction warnings generated daily by modern mega-constellations, and communication blackout zones leave satellites entirely unprotected during critical windows.

This project presents a full-stack Autonomous Space Situational Awareness (SSA) Dashboard built for the National Space Hackathon 2026, hosted by IIT Delhi. The system simulates a Walker Delta constellation of 6 satellites at 550 km altitude tracking 518 debris objects in real time, powered by a physics-accurate 4th-order Runge-Kutta (RK4) propagator with J2 perturbation corrections. The Collision Avoidance (COLA) engine autonomously detects threats, generates CDMs with probability of collision, monitors Time to Closest Approach (TCA), and fires evasion and recovery burns without human intervention when TCA falls below 5 hours.

The platform features a 3D WebGL globe rendered with Three.js, a 2D Mercator ground track map with NASA Blue Marble imagery and live terminator overlay, a Conjunction Bullseye Plot, Proximity Operations Radar, Manoeuvre Gantt Timeline, and Fleet Resource Dashboard. A global network of 40 ground stations is modelled for line-of-sight communication constraints. The backend is built on FastAPI with a PostgreSQL (Neon) database, and the frontend is built on React 18 with Vite, deployed on Vercel.

KEYWORDS: Orbital Debris Avoidance, Collision Avoidance (COLA), Space Situational Awareness (SSA), Conjunction Data Message (CDM), RK4 Orbital Propagator, Walker Delta Constellation, Three.js WebGL, FastAPI, LEO.

Acknowledgement

We extend our heartfelt appreciation to all those who contributed to the development and success of this project.

We express our sincere gratitude to our Project Supervisor Mr. G. Narasimham and the Department of Computer Science and Engineering, JNTUH, for their valuable guidance and continuous support throughout the project.

We express our deepest gratitude to our faculty members whose guidance and mentorship were invaluable throughout the development process. Their expertise in both software engineering and systems design provided us with the direction needed to navigate the complex challenges of real-time orbital simulation, physics engine development, and full-stack deployment.

We acknowledge the inspiration drawn from the National Space Hackathon 2026 problem statement issued by the Indian Institute of Technology, Delhi. The challenge framing centred around the Kessler Syndrome, autonomous constellation management, and scalable collision avoidance provided a clear and compelling technical vision for our system.

We are grateful for the open-source technologies that formed the foundation of this project: React, Vite, Three.js, FastAPI, Python, and PostgreSQL. The broader open-source community's documentation, examples, and tooling made it possible to implement complex orbital mechanics and real-time WebGL rendering within a hackathon timeframe.

Finally, we thank NASA for publicly available orbital mechanics references and the Blue Marble Earth imagery that brought our 2D ground track visualization to life.

Table of Contents

Abstract 3
Acknowledgement 4
1. Introduction 7
   1.1 Motivation
   1.2 Problem Statement
   1.3 Project Objectives
2. Requirement Analysis 10
   2.1 Functional Requirements
   2.2 Non-Functional Requirements
   2.3 Software Requirements
   2.4 Hardware Requirements
3. UML Diagrams 14
   3.1 UML Diagram
   3.2 Class Diagram
4. Project Design and Development 16
   4.1 Development Methodology
   4.2 Physics Engine & Orbital Mechanics
   4.3 COLA Engine
   4.4 Visualization Modules
5. Tools and Technologies 19
   5.1 Frontend Development
   5.2 Backend Development
   5.3 Database
6. Project Phases 21
7. Testing 23
   7.1 Unit Testing
   7.2 Integration Testing
   7.3 Functional Testing
   7.4 Performance Testing
   7.5 Usability Testing
   7.6 Regression Testing
   7.7 User Acceptance Testing (UAT)
8. User Guide 26

9. Conclusion 29
10. References 30
11. Appendix 31

1. Introduction

1.1 Motivation

The rapid proliferation of satellites and space debris in Low Earth Orbit (LEO) has transformed what was once an open frontier into a congested and increasingly hazardous environment. Commercial mega-constellations now operate alongside millions of uncontrolled debris fragments defunct rocket bodies, shattered solar panels, and stray hardware all travelling at hypervelocity speeds exceeding 27,000 km/h. This severe congestion brings the global space community dangerously close to the Kessler Syndrome, a cascading collision scenario theorized by NASA scientist Donald Kessler, where a single impact generates shrapnel that triggers further collisions in an unstoppable chain reaction.

Currently, satellite collision avoidance is a manual, ground-reliant process. Flight Dynamics Officers (FDOs) receive Conjunction Data Messages (CDMs) from radar networks such as the US Space Surveillance Network, evaluate threats by hand, and uplink thruster commands to satellites. This human-in-the-loop model, while functional for small fleets, is fundamentally unsustainable as constellation sizes scale to thousands of satellites generating hundreds of conjunction warnings every single day.

Three critical bottlenecks make the status quo untenable: manual evaluation cannot scale, satellites regularly pass through communication blackout zones over oceans and poles where no ground station has line-of-sight, and human operators are ill-equipped to globally optimize fuel consumption across an entire fleet in real time. The clear need for an autonomous, intelligent, and scalable solution was the primary motivation behind this project.

1.2 Problem Statement

The absence of an autonomous, high-performance collision avoidance system for LEO constellations presents a critical and multi-dimensional challenge for the space industry. Key issues include:

- **Scalability of Manual Operations:** Manual evaluation of CDMs cannot scale to handle constellations comprising thousands of satellites, which may collectively face hundreds of conjunction warnings daily.
- **Communication Blackouts and Blind Conjunctions:** Satellites regularly pass over deep oceans, polar regions, and other areas beyond the coverage of any ground station. If a conjunction is predicted during

such a blackout, ground control is entirely unable to uplink an evasion command.

- **Suboptimal Propellant Management:** Spacecraft fuel is finite and non-replenishable. Human operators lack the computational capacity to globally minimize delta-V expenditure across a full fleet while simultaneously ensuring every satellite returns to its mission orbital slot.
- **O(N²) Computational Bottleneck:** Naively checking every satellite against every debris object for collision risk is an O(N²) operation. With 518+ tracked debris objects and a growing constellation, this approach is computationally prohibitive at real-time update frequencies.
- **No Automated Station-Keeping Recovery:** Evasion manoeuvres inherently perturb a satellite's orbit. Without an automated recovery burn system, a satellite displaced from its nominal slot will remain in service outage indefinitely, degrading constellation coverage.

1.3 Project Objectives

The primary objectives of this project are:

- To develop a full-stack Autonomous Space Situational Awareness (SSA) Dashboard providing real-time orbital simulation, threat visualization, and mission-critical decision support for a 6-satellite LEO constellation.
- To implement a physics-accurate orbital propagation engine using a 4th-order Runge-Kutta (RK4) integrator with J2 perturbation corrections, tracking 6 active satellites and 518 debris objects simultaneously.
- To build an autonomous Collision Avoidance (COLA) engine that detects conjunction threats, generates CDMs with probability of collision, monitors Time to Closest Approach (TCA), and automatically fires evasion and recovery burns when TCA falls below 5 hours.
- To create a 3D WebGL visualization globe using Three.js with a real Earth texture, day/night cycle, live satellite markers, orbital rings, and debris cloud rendering, alongside a 2D Mercator ground track map with terminator overlay.
- To implement a comprehensive mission analytics dashboard including a Conjunction Bullseye Plot, Proximity Operations Radar, Manoeuvre Gantt Timeline, and Fleet Resource/Propellant tracking panel.
- To integrate a 40-station global ground network (NASA DSN, ESA ESTRACK, ISRO, JAXA, CNSA) for simulating communication coverage, blackout zones, and line-of-sight constraints on manoeuvre scheduling.

- To deploy a RESTful FastAPI backend with a PostgreSQL database (Neon) handling telemetry ingestion, simulation stepping, manoeuvre scheduling, and CDM/operations logging.
- To build a scalable, responsive React 18 + Vite frontend deployable on Vercel, providing an operational interface accessible across desktop, tablet, and mobile devices.

2. Requirement Analysis

The requirement analysis phase is crucial for defining the scope and functionalities of the project. Requirements are divided into functional and non-functional categories to ensure all aspects of the system are covered.

2.1 Functional Requirements

Orbital Simulation & Physics Engine

- The system must simulate a Walker Delta constellation of 6 satellites at 550 km altitude, 51.6° inclination, using an RK4 integrator with J2 perturbation corrections.
- The simulation must track 518 independent debris objects, each with their own orbital parameters, propagated in real time.
- The backend must expose a POST /api/simulate/step endpoint accepting a step_seconds parameter to advance the simulation by arbitrary time increments.
- The physics engine must model the Tsiolkovsky rocket equation for fuel mass depletion on every burn, dynamically updating satellite wet mass after each manoeuvre.

Telemetry Ingestion

- The system must accept high-frequency state vector updates (position and velocity in ECI frame) via a POST /api/telemetry endpoint.
- Telemetry payloads must support both satellite and debris object types, identified by a unique ID and object type field.
- The backend must process incoming telemetry asynchronously and return an acknowledgement with the count of active CDM warnings.

Conjunction Assessment & COLA Engine

- The system must predict potential collisions up to 24 hours ahead and generate Conjunction Data Messages with probability of collision for each identified threat.
- A conjunction is defined as any predicted miss distance below 100 metres between a satellite and a debris object.
- When TCA falls below 5 hours for any active threat, the COLA engine must autonomously fire an evasion burn without requiring human input.
- Every evasion burn must be paired with a recovery burn that returns the satellite to within 10 km of its nominal orbital slot.

Manoeuvre Scheduling

- The system must expose a POST /api/manoeuvre/schedule endpoint that validates ground station line-of-sight, fuel sufficiency, and thruster cooldown constraints before accepting a burn command.
- A mandatory 600-second cooldown must be enforced between any two consecutive burns on the same satellite.
- Individual burn delta-V magnitude must not exceed 15.0 m/s per command.
- All Manoeuvre events must be logged to the database with timestamps, satellite ID, burn type, and delta-V vector.

Visualization & Dashboard

- The frontend must render a 3D WebGL Earth globe using Three.js, displaying all 6 satellite positions, orbital rings, 518 debris points, and threat indicators in real time.
- A 2D Mercator ground track map must display sub-satellite points, historical trailing paths, predicted trajectories, and a dynamic day/night terminator line.
- A Conjunction Bullseye Plot must show approaching debris relative to a selected satellite, with color-coded risk levels (Green = Safe, Yellow < 5 km, Red < 1 km).
- A Manoeuvre Gantt Timeline must display evasion and recovery burn blocks, cooldown windows, and blackout zone overlaps on a dynamic time axis.
- A Fleet Resource Dashboard must show per-satellite propellant bars, delta-V cost graphs, and mission budget tracking.

Threat Simulation & Alerts

- The system must support manual injection of a debris object targeting any selected satellite, with a 5-second countdown before activation.
- A full-screen pulsing collision warning alert must appear at threat activation, with an ACKNOWLEDGE button to dismiss.
- A toast notification system must display real-time event updates for all simulation actions.

2.2 Non-Functional Requirements

- **Performance:** The 3D WebGL globe must maintain a stable 60 FPS while rendering 6 satellites and 518 debris objects simultaneously.

- **Performance:** The /api/visualization/snapshot endpoint must return a compressed, flattened JSON payload (debris cloud as tuples) to minimize network transfer time.
- **Usability:** The interface must follow a consistent dark-themed glassmorphism design language with clear visual hierarchy, and provide an interactive 7-step tutorial modal.
- **Usability:** The application must be fully responsive across desktop, tablet, and smartphone viewports.
- **Security:** The backend API must validate all incoming telemetry and Manoeuvre payloads, rejecting malformed or out-of-range values.
- **Security:** PostgreSQL credentials must be managed via environment variables and never exposed in frontend-accessible code.
- **Security:** All physical constraints (fuel levels, cooldown periods, delta-V limits) must be enforced server-side.
- **Reliability:** The simulation state must be recoverable from the database after a backend restart, ensuring no mission data is lost.
- **Scalability:** The backend must be designed to support scaling from 6 to 50+ satellites and 518 to 10,000+ debris objects with spatial indexing.

2.3 Software Requirements

Category	Requirement
Operating System	Windows 10+, macOS, Ubuntu Linux
Runtime (Backend)	Python 3.10+
Runtime (Frontend)	Node.js 18+
Web Browser	Latest Chrome, Firefox, Safari, Edge (WebGL support required)
Database	PostgreSQL (Neon cloud or Supabase)
Dev Tools	VS Code, Git, Postman (API testing), PowerShell (Windows launch scripts)
Deployment	Vercel (frontend), Uvicorn ASGI server (backend)

2.4 Hardware Requirements

Client-Side

- Processor: Intel Core i5 or equivalent (required for WebGL rendering)

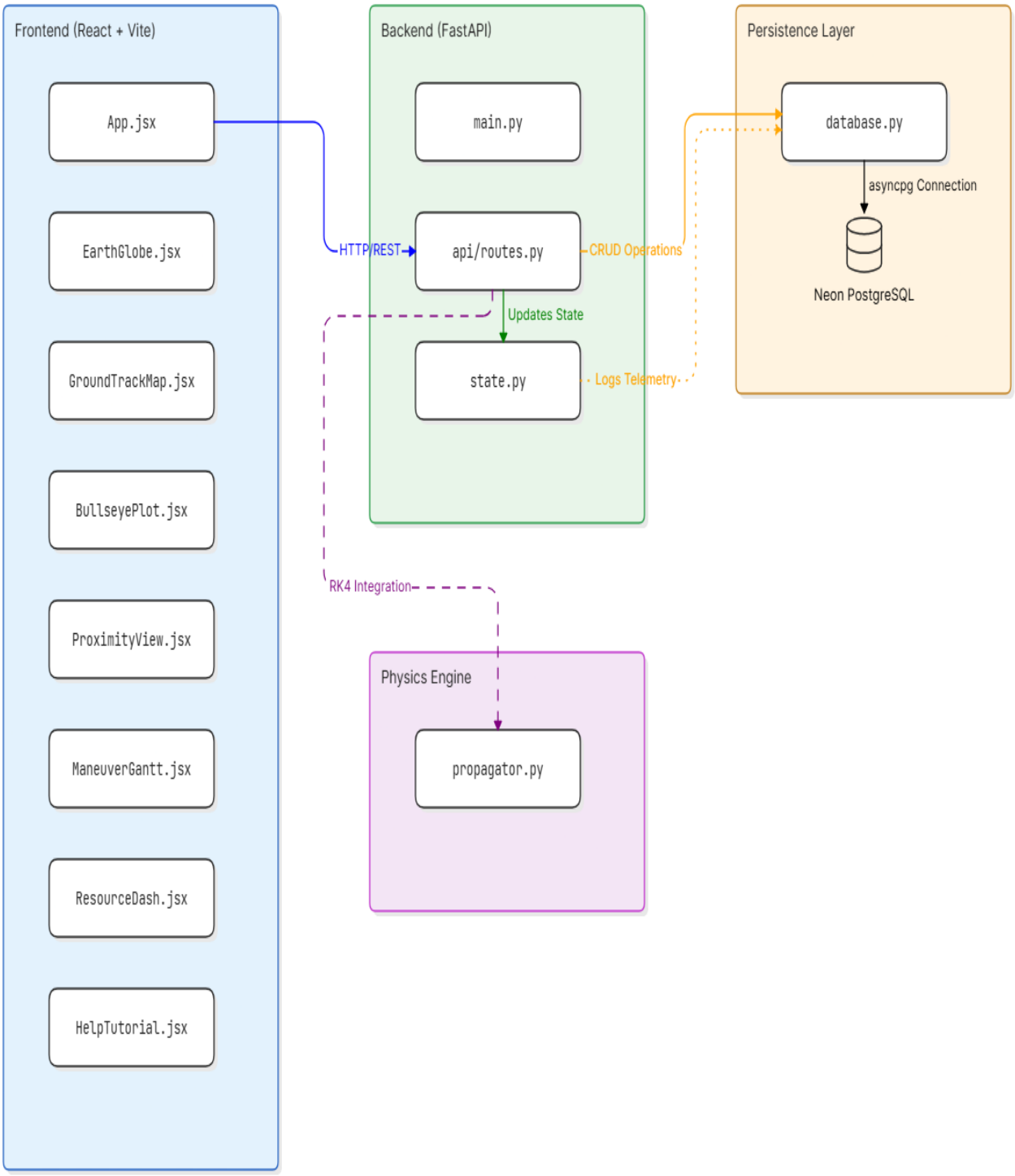
- RAM: 8 GB or more
- GPU: Dedicated or integrated GPU with WebGL 2.0 support
- Active Internet Connection

Server-Side

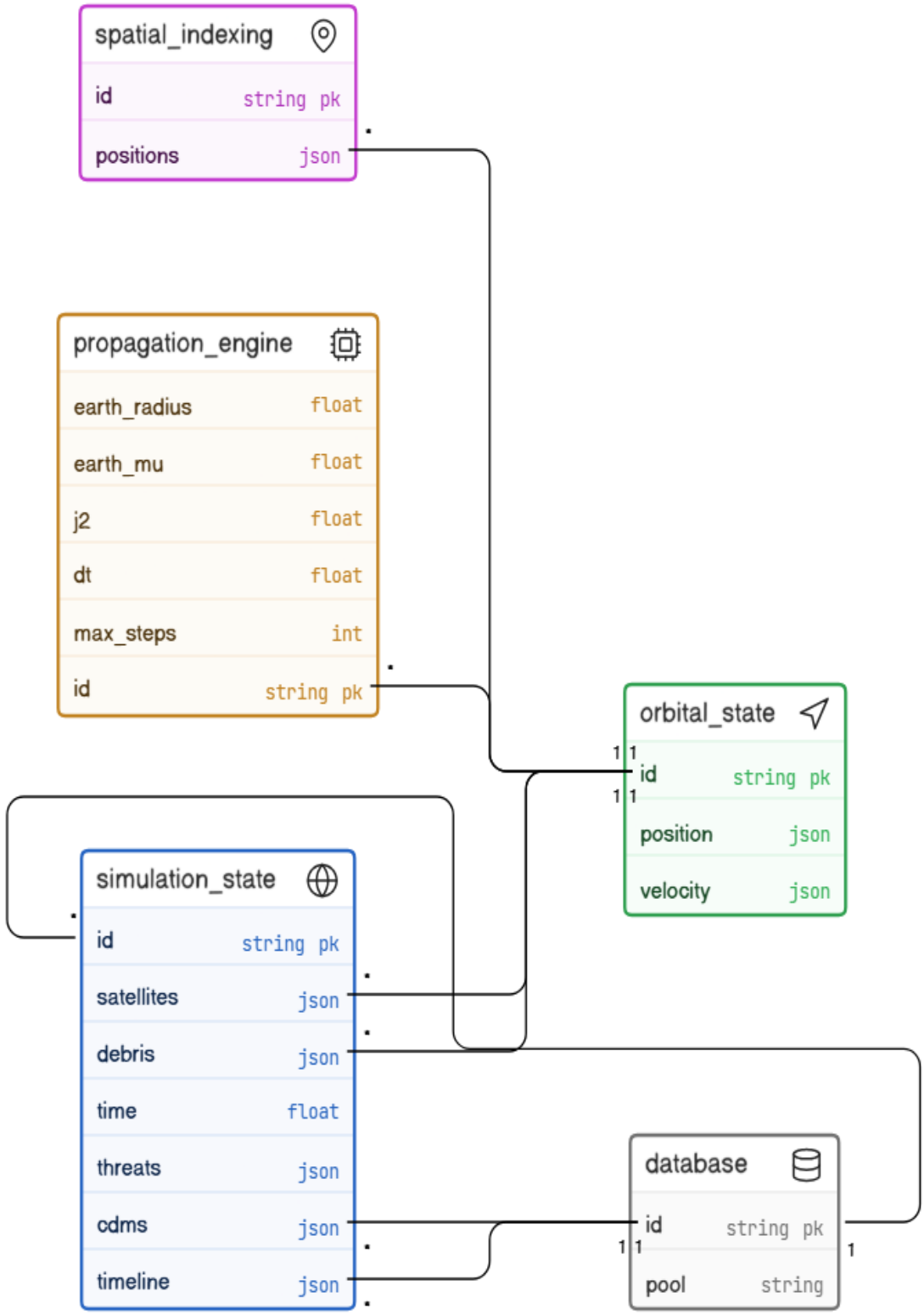
- Processor: 2-core CPU minimum
- RAM: 4 GB or more
- Storage: 20 GB SSD minimum for application files and database

3. UML Diagrams

3.1 Use Case Diagram



3.2 Class Diagram



4. Project Design and Development

4.1 Development Methodology

For the development of this system, a hybrid approach was employed that strategically combines the structured planning of the Waterfall model with the iterative flexibility of Agile methodologies. This allowed the project to be built on a well-planned foundation while remaining adaptable to the unique technical challenges of real-time orbital simulation.

- **Waterfall Elements:** The initial phases — requirements gathering, system design, API specification, and database schema design — were conducted sequentially, following the NSH problem statement as a formal specification document.
- **Agile Elements:** The implementation phase was executed using an iterative, sprint-like approach. The project was broken into focused modules (physics engine, COLA engine, API layer, visualization components), each developed, tested, and integrated in cycles.

Sprint Breakdown:

- Sprint 1 — Backend core: FastAPI setup, database connection, simulation state management, and RK4 propagator implementation.
- Sprint 2 — Physics validation: J2 perturbation integration, debris cloud initialization, and Walker Delta constellation setup.
- Sprint 3 — COLA engine: Conjunction detection algorithm, CDM generation, TCA monitoring, and autonomous evasion/recovery burn scheduling.
- Sprint 4 — Frontend structure: React component architecture, Three.js globe, and 2D Mercator map.
- Sprint 5 — Visualization modules: Bullseye Plot, Proximity Radar, Manoeuvre Gantt, Resource Dashboard, and threat alert system.
- Sprint 6 — Integration & polish: Ground station LOS validation, 7-step tutorial, responsive design, and Vercel deployment.

4.2 Physics Engine & Orbital Mechanics

The orbital mechanics core is the most technically demanding component of the system. Key implementation details include:

- **State Vectors:** All satellite and debris objects are represented as 6-dimensional state vectors [x, y, z, vx, vy, vz] in the Earth-Centered Inertial (ECI) frame (J2000 epoch).
- **J2 Perturbation:** The equations of motion incorporate both the standard two-body gravitational term and the J2 perturbation acceleration, which accounts for Earth's equatorial bulge causing nodal regression and apsidal precession.
- **RK4 Integrator:** A 4th-order Runge-Kutta integrator propagates state vectors forward in time with high numerical accuracy. The RK4 method evaluates the derivative at four intermediate points per step, achieving 4th-order convergence.
- **Fuel Mass Depletion:** Fuel depletion is modelled using the Tsiolkovsky rocket equation. Each burn decreases the satellite's propellant mass, and subsequent Manoeuvres account for the reduced wet mass, making later burns slightly more efficient.
- **Constellation Parameters:** 6 satellites in a Walker Delta 6/6/1 formation at 550 km altitude, 51.6° inclination. Orbital period: ~95.6 minutes. Velocity: ~7.58 km/s. Initial propellant: 50 kg hydrazine per satellite.

4.3 COLA Engine

The Collision Avoidance engine operates as a continuous background process:

- **Threat Detection:** Each simulation step evaluates the predicted closest approach distance between every satellite and every tracked debris object using propagated state vectors.
- **CDM Generation:** When predicted miss distance falls below 100 metres, a CDM is generated recording the satellite ID, debris ID, TCA timestamp, miss distance, and probability of collision.
- **TCA Monitoring:** TCA is continuously monitored for all active CDMs. When TCA falls below the 5-hour threshold, the COLA engine autonomously schedules an evasion burn.
- **Evasion Burn Calculation:** Evasion burns are calculated in the RTN (Radial-Transverse-Normal) frame. Prograde/retrograde burns along the Transverse axis are preferred as they are the most fuel-efficient method for changing orbital phasing.
- **Recovery Burns:** After the debris threat passes safely, the engine calculates and schedules a recovery burn to return the satellite to within 10 km of its nominal orbital slot, minimizing service outage time.

- **Fuel Budget:** Each evasion burn costs approximately 2.5 kg of propellant. The system tracks remaining fuel for all satellites and flags any satellite whose reserves fall below a critical threshold for graveyard orbit decommissioning.

4.4 Visualization Modules

The dashboard provides six distinct visualization panels, each targeting a specific aspect of situational awareness:

Module	Technology	Purpose
3D Earth Globe	Three.js WebGL	Real-time 3D orbital view with day/night cycle and debris cloud
Ground Track Map	SVG + NASA Blue Marble	2D Mercator sub-satellite point tracking with terminator line
Bullseye Plot	SVG Polar Chart	Relative proximity view of conjunction threats with TCA rings
Proximity Radar	SVG	Nearby object radar with TCA countdown and risk color coding
Manoeuvre Gantt	SVG Timeline	Chronological burn schedule with cooldown and blackout blocks
Resource Dashboard	SVG Bar Charts	Fleet-wide propellant levels and delta-V cost analysis

5. Tools and Technologies

The project utilises a modern full-stack technology suite selected to meet the demanding performance, accuracy, and real-time rendering requirements of an orbital simulation dashboard.

5.1 Frontend Development

- **React 18 + Vite:** React 18 is the core UI framework, providing a component-based architecture for building the complex, multi-panel dashboard. Hooks such as useState, useEffect, and useRef manage simulation state and animation loops. Vite serves as the build tool, providing near-instant hot module replacement during development.
- **Three.js:** Three.js is used to render the 3D WebGL Earth globe. It handles texture mapping (real Earth imagery), directional lighting (sun simulation), OrbitControls for interactive rotation, and the rendering of satellite markers, orbital rings, and the 518-point debris cloud — all targeting 60 FPS.
- **SVG Visualization:** All 2D visualizations including the Mercator ground track map, Bullseye Plot, Proximity Radar, Manoeuvre Gantt, and Resource Dashboard are built using raw SVG elements manipulated directly by React. This approach provides complete control over rendering performance without additional charting library overhead.
- **Vanilla CSS:** All styling is implemented in Vanilla CSS using a glassmorphism design language with dark backgrounds, frosted-glass panel effects, and smooth CSS animations for the threat alert pulsing and shake effects.

5.2 Backend Development

- **FastAPI + Uvicorn:** FastAPI is the Python web framework used to build the RESTful API. It provides automatic OpenAPI documentation, async request handling via Uvicorn, and Pydantic-based request validation. Its async capabilities are essential for non-blocking telemetry ingestion while the physics engine runs concurrently.
- **Python Physics Engine:** The physics engine is implemented in pure Python using standard numerical methods. The RK4 integrator, J2 perturbation model, Tsiolkovsky fuel equation, and conjunction detection algorithms are all implemented in routes.py and state.py, without external orbital mechanics libraries.

- **Async Architecture:** All computation-heavy operations (state propagation, conjunction screening) are designed to run asynchronously, ensuring that high-frequency telemetry ingestion does not block the visualization snapshot endpoint.
- **Environment Management:** Python-dotenv manages environment variables for database credentials. The requirements.txt file pins all dependency versions for reproducible deployment.

5.3 Database

- **PostgreSQL (Neon):** PostgreSQL hosted on Neon (serverless cloud Postgres) is used as the persistent data store. The asyncpg driver enables non-blocking database operations from the FastAPI async context. The schema is initialized via seed.sql and ground station data is loaded via a dedicated Python script.

Tool / Library	Version	Purpose
React	18.x	Frontend UI framework
Vite	Latest	Frontend build tool and dev server
Three.js	r128	3D WebGL rendering
FastAPI	Latest	Python async REST API framework
Uvicorn	Latest	ASGI server for FastAPI
PostgreSQL	Neon Cloud	Persistent relational database
asyncpg	Latest	Async PostgreSQL driver for Python
Python	3.10+	Backend runtime
Node.js	18+	Frontend runtime and build
Vercel	—	Frontend deployment platform
Git + GitHub	—	Version control and repository hosting

6. Project Phases

The development was organized into four structured phases, ensuring a systematic progression from concept to a fully functional deployed application.

Phase 1: Initiation and Planning

- **Idea Finalization:** Problem Statement Analysis: The NSH 2026 problem statement was analysed in depth to extract the core technical requirements — physics accuracy, API specifications, visualization modules, evaluation criteria, and deployment constraints.
- **Technology Research:** Technology Selection: React + Vite for the frontend (chosen for component architecture and WebGL integration via Three.js), FastAPI for the backend (chosen for async performance and auto-validation), and Neon PostgreSQL (chosen for serverless scalability).
- **Requirement Definition:** API Contract Definition: All API endpoints, request/response schemas, and database table structures were defined before any code was written, following the NSH specification.
- **Workload Division:** Workload Division: Frontend visualization, backend physics engine, COLA logic, and database/DevOps responsibilities were assigned to team members based on individual strengths.

Phase 2: Design and Prototyping

- **Architecture Design:** High-Level Architecture: A client-server architecture diagram was designed establishing the React frontend, FastAPI backend, and PostgreSQL database layers with clear API boundaries.
- **Schema Design:** Database Schema: The five-table schema (telemetry, Manoeuvre_events, cdm_log, operations_log, ground_stations) was designed and implemented in seed.sql.
- **UI Prototyping:** UI/UX Layout: Component wireframes were created for all six visualization modules. The dark glassmorphism theme was established early to ensure a consistent visual identity across all panels.
- **Physics Validation:** Physics Model Validation: The J2 perturbation equations, RK4 integration scheme, and Tsiolkovsky rocket equation were validated analytically before implementation.

Phase 3: Execution (Development Phase)

- Backend core was built first — FastAPI app, database connection, simulation state manager, and the RK4 propagator with J2 corrections.
- The COLA engine was implemented and tested in isolation, verifying correct CDM generation, TCA tracking, and autonomous burn scheduling logic.
- The Three.js globe and SVG Mercator map were developed in parallel with the backend, initially consuming mock data before full API integration.
- All five remaining visualization modules (Bullseye, Proximity, Gantt, Resource, Tutorial) were implemented iteratively, each wired to live API data on completion.
- Ground station network data was loaded from CSV, and the line-of-sight validation logic was integrated into the Manoeuvre scheduling endpoint.
- The random threat simulation, collision warning alert system, and toast notification queue were added as the final frontend features.

Phase 4: Monitoring, Control & Deployment

- Continuous unit and integration testing was performed throughout development, with Postman used to validate all API endpoints against the NSH specification.
- End-to-end testing verified the complete threat lifecycle: debris injection → CDM generation → COLA auto-evasion → recovery burn → station-keeping return.
- Performance profiling identified and resolved rendering bottlenecks in the Three.js debris cloud renderer.
- The frontend was deployed to Vercel with a vercel.json proxy configuration routing /api/* requests to the FastAPI backend.
- A PowerShell launch script (install_Launch.ps1) was created to enable one-command local setup and launch of both frontend and backend servers.

7. Testing

Testing was a critical and continuous phase ensuring that every component of the system functioned correctly in isolation and as part of the integrated simulation platform. A multi-layered testing strategy was adopted.

7.1 Unit Testing

Unit testing focused on testing individual components and functions in isolation.

- Physics Functions: The RK4 integrator was validated by propagating a satellite orbit for one full period and verifying that the resulting position matched the expected value within acceptable numerical tolerance.
- J2 Perturbation: The J2 acceleration vector computation was validated against reference values from orbital mechanics literature.
- Tsiolkovsky Equation: Fuel mass depletion calculations were verified for known delta-V inputs and initial masses, confirming correct exponential mass decay.
- CDM Generation: The conjunction detection function was tested with synthetic satellite and debris state vectors at known separations, confirming correct triggering at the 100-metre threshold.

7.2 Integration Testing

Integration testing verified that different system modules worked together correctly.

- Telemetry-to-State Integration: POST /api/telemetry payloads were verified to correctly update the internal simulation state, confirming that debris state vectors were stored and accessible to the conjunction screening loop.
- COLA-to-Manoeuvre Integration: When the COLA engine triggered an auto-evasion burn, the resulting Manoeuvre was verified to be correctly logged in the Manoeuvre_events table and returned in subsequent /api/visualization/snapshot responses.
- Frontend-Backend Integration: Postman was used to manually exercise all API endpoints before connecting the React frontend, verifying correct JSON response schemas.
- Database Persistence: After a simulated backend restart, mission state was verified to be correctly restored from the PostgreSQL database.

7.3 Functional Testing

Functional testing validated the complete user-facing feature set:

- Full threat lifecycle test: debris injection → 5-second countdown → collision warning alert → CDM generation → COLA auto-evasion → recovery burn → station-keeping return.
- Satellite fuel depletion: repeated evasion burns were triggered to verify correct fuel tracking and DIED status display on the ground track map.
- Ground station LOS: Manoeuvre commands were submitted while a satellite was in a simulated blackout zone, verifying correct rejection by the API.
- Simulation step API: the /api/simulate/step endpoint was called with varying step_seconds values, verifying correct physics advancement and timestamp progression.

7.4 Performance Testing

- **Rendering Performance:** Three.js WebGL rendering was profiled using browser developer tools. Frame rate was measured with all 518 debris points active, confirming stable 60 FPS on target hardware.
- **API Response Time:** The /api/visualization/snapshot endpoint was benchmarked to verify low-latency JSON response times suitable for real-time frontend polling intervals.
- **Physics Engine Speed:** The RK4 integrator was timed for a full simulation step over all 524 tracked objects (6 satellites + 518 debris), confirming sub-second computation time.

7.5 Usability Testing

- **Tutorial Effectiveness:** The 7-step interactive tutorial was tested with users unfamiliar with the system, confirming that all major features were discoverable and understandable within the guided walkthrough.
- **Responsive Design:** The responsive layout was tested across desktop (1920x1080), tablet (768px), and mobile (375px) viewports, verifying usable panel layouts at all breakpoints.
- **Alert Visibility:** The collision warning alert system was tested for visibility and clarity, confirming that the full-screen pulsing animation and ACKNOWLEDGE button were immediately noticeable during active threat scenarios.

7.6 Regression Testing

After each major feature addition, a regression suite of core functional tests was re-run to verify that new changes did not break previously working features. Key regression tests included the simulation step cycle, COLA auto-evasion trigger, visualization snapshot schema, and database logging.

7.7 User Acceptance Testing (UAT)

A final demonstration of the fully integrated system was conducted, validating the complete feature set against the NSH 2026 evaluation criteria. The demo covered the 3D globe rendering, COLA auto-evasion lifecycle, all six visualization modules, ground station network, threat simulation, and the mission reporting panel. All originally specified objectives were confirmed as met.

8. User Guide

This guide provides a step-by-step walkthrough of the key features of the Orbital Debris Avoidance & Constellation Management System dashboard.

8.1 Landing Page

The animated landing page is the entry point to the system. A CSS-animated satellite graphic introduces the project theme. Clicking Launch Dashboard transitions to the main SSA Dashboard. The landing page communicates the project title and hackathon context at a glance.

8.2 3D Earth Globe

The globe panel provides a real-time three-dimensional view of the orbital environment.

- Six satellite markers orbit the Earth on visible orbital ring paths, color-coded by status (active vs. fuel-depleted).
- 518 debris points are rendered as small red markers distributed across LEO.
- Active conjunction threats are highlighted with a visual threat indicator connecting the satellite to the approaching debris.
- The DirectionalLight (sun) rotates in real time based on simulation clock, producing a live day/night cycle on the Earth texture.
- The globe supports interactive rotation, zoom, and pan via Three.js OrbitControls.

8.3 Ground Track Map (2D Mercator)

The 2D map panel displays sub-satellite points over a NASA Blue Marble Earth background.

- Live satellite position markers are plotted in real time as the simulation advances.
- A dynamic terminator line with 23.5° axial tilt overlay separates the sunlit and shadow hemispheres.
- Satellite status labels (SAFE / DIED) are displayed based on remaining fuel level.
- 40 global ground station markers are shown, enabling visual assessment of communication coverage.

8.4 Conjunction Bullseye Plot

The Bullseye Plot provides a relative proximity view of debris threats approaching a selected satellite.

- The selected satellite is fixed at the centre of the polar chart.
- Debris objects are plotted with radial distance representing Time to Closest Approach (TCA).
- Approach angle represents the relative vector of the incoming debris.
- Color coding: Green = Safe, Yellow = Warning (< 5 km), Red = Critical (< 1 km).

8.5 Proximity Operations Radar

The Proximity Radar displays a wider view of all objects within a defined range of the selected satellite, with TCA countdown timers and risk level classification rendered as concentric color-coded rings.

8.6 Manoeuvre Gantt Timeline

The Gantt Timeline provides a chronological schedule of all past and upcoming automated Manoeuvres.

- Evasion burn blocks are shown in red.
- Recovery burn blocks are shown in blue.
- Mandatory 600-second thruster cooldown periods are shown in grey.
- The timeline window dynamically adjusts to keep the current simulation time centred.

8.7 Fleet Resource Dashboard

The Resource Dashboard provides fleet-wide health monitoring at a glance.

- Propellant bars display the remaining fuel_kg for each of the 6 satellites, updating after every burn.
- A delta-V cost bar chart plots total fuel consumed versus conjunctions avoided, visually demonstrating evasion efficiency.
- Mission budget summary shows total burns executed, total delta-V spent, and estimated remaining operational lifetime per satellite.

8.8 Random Threat Simulation

Operators can manually inject a debris threat at any time using the threat simulation controls.

- Select a target satellite (alpha-01 through alpha-06) from the dropdown.

- Click Spawn Threat to inject a debris object on a collision course with the selected satellite.
- A 5-second countdown begins, after which a full-screen pulsing COLLISION WARNING alert activates with a red shake animation.
- Click ACKNOWLEDGE to dismiss the alert. The COLA engine will autonomously handle the evasion sequence.

8.9 Interactive Tutorial

First-time users can launch the 7-step guided tutorial from the Help button. The tutorial walks through each panel of the dashboard in sequence, highlighting key features and explaining the simulation controls, threat lifecycle, and visualization panels.

8.10 Reports & Mission Summary

The Reports panel provides a live mission summary including current fleet status (fuel levels, Manoeuvre counts, SAFE/DIED classification), full CDM log history with timestamps and miss distances, and a complete Manoeuvre execution history with burn type, satellite ID, and delta-V cost.

9. Conclusion

The Orbital Debris Avoidance & Constellation Management System successfully demonstrates a complete, full-stack implementation of an Autonomous Space Situational Awareness platform.

The project directly addresses the core challenges of modern LEO congestion management: scalability beyond manual human-in-the-loop operations, autonomous response during communication blackout windows, physically accurate orbital mechanics using RK4 integration with J2 perturbation corrections, and fuel-aware Manoeuvre optimization guided by the Tsiolkovsky rocket equation.

The system achieves its core objectives across three dimensions. On the physics layer, a rigorous orbital mechanics engine tracks 524 objects simultaneously, enforces all spacecraft physical constraints, and executes the complete COLA lifecycle from conjunction detection through CDM generation, autonomous evasion burn, and station-keeping recovery without human intervention. On the visualization layer, six purpose-built dashboard modules provide genuine situational awareness comparable to professional Flight Dynamics Officer workstations, powered by a Three.js WebGL 3D globe and five SVG-based analytical panels. On the infrastructure layer, a FastAPI backend with PostgreSQL persistence and a Vercel-deployed React frontend deliver a production-grade full-stack deployment.

The platform is architecturally positioned to scale to the 50+ satellite and 10,000+ debris object requirements. Future work would focus on implementing spatial indexing algorithms (such as octrees or k-d trees) to eliminate the $O(N^2)$ conjunction screening bottleneck, integrating real Two-Line Element (TLE) data from Space-Track.org, and deploying the backend with a Dockerfile for automated grading compatibility.

10. References

1. National Space Hackathon 2026 — Official Problem Statement. Indian Institute of Technology, Delhi. 2026.
2. Kessler, D. J. and Cour-Palais, B. G. — Collision Frequency of Artificial Satellites: The Creation of a Debris Belt. Journal of Geophysical Research, 1978.
3. Vallado, D. A. — Fundamentals of Astrodynamics and Applications. 4th Edition. Microcosm Press, 2013.
4. Three.js Official Documentation. https://threejs.org/docs/
5. FastAPI Official Documentation. https://fastapi.tiangolo.com/
6. React Official Documentation. https://react.dev/
7. PostgreSQL Official Documentation. https://www.postgresql.org/docs/
8. NASA Blue Marble Next Generation — Earth Observatory. https://earthobservatory.nasa.gov/features/BlueMarble
9. US Space Surveillance Network — Space-Track.org. https://www.space-track.org/
10. Neon Serverless Postgres Documentation. https://neon.tech/docs/

11. Appendix

Source Code Repository

The complete source code for the project is hosted on GitHub:

https://github.com/AridoshikaZu103/Orbital-Debris-Avoidance-Constellation-Management-System

API Endpoint Summary

Method	Endpoint	Description
GET	/api/visualization/snapshot	Returns current state: satellites, debris, threats, timeline
POST	/api/simulate/step	Advances simulation by step_seconds
POST	/api/telemetry	Injects telemetry state vectors for satellites or debris
POST	/api/Manoeuvre/schedule	Schedules an evasion or recovery burn for a satellite
GET	/api/health	Backend health check endpoint

Constellation Parameters

Parameter	Value
Formation	Walker Delta 6/6/1
Altitude	550 km (LEO)
Inclination	51.6°
Orbital Period	~95.6 minutes
Velocity	~7.58 km/s
Propellant	50 kg hydrazine per satellite
Dry Mass	500 kg per satellite
Specific Impulse (Isp)	300.0 s
Max Delta-V per burn	15.0 m/s
Thruster Cooldown	600 seconds
Debris Tracked	518 objects

Parameter	Value
Propagator	RK4 (4th-order Runge-Kutta) + J2
Conjunction Threshold	100 metres (0.1 km)
Station-Keeping Radius	10 km spherical

Ground Station Network Sample

Station ID	Name	Latitude	Longitude	Min Elevation
GS-001	ISTRAC, Bengaluru	13.03° N	77.51° E	5.0°
GS-002	Svalbard Satellite Station	78.22° N	15.41° E	5.0°
GS-003	Goldstone Tracking, California	35.43° N	116.89° W	10.0°
GS-004	Punta Arenas, Chile	53.15° S	70.92° W	5.0°
GS-005	IIT Delhi Ground Node	28.54° N	77.19° E	15.0°
GS-006	McMurdo Station, Antarctica	77.85° S	166.67° E	5.0°